DATA Bricks

Reading data :-

1. df = spark.read.csv('Path') -> It is for csv file, when you are using catalog we have to replace .csv as .table like df = spark.read.table('Path')
2. df.show() -> like print It shows the data
3. display(df) -> It is another way.
4. df.printschema() -> To know schema.
5. df = spark.read.csv('path', header=True, inferSchema=True) (CAP)
6. df = spark.read.format("file_type").
   option(header=True, inferschema=True).
   load('path').

This is for all type files.

Select

1. df.select("col1", "col2", ...).show()
2. df.select(df.columns[1:5]).show()
3. df.select(*df.columns)

withColumn() & withColumnRenamed()

withColumn() -> Change value, Change datatype existing column, create a new row & many more.

withColumnRenamed() -> Rename of column. (CAP)

Ex: df.withColumn("column_name", condition).show() -> New column. (CAP)

df.withColumn("salary1", df.salary*10).show().

df = df.withColumn("column_name", col("column_name").cast("Integer")). (CAP)

This is for Data type changing for columns.

df.withColumnRenamed("old_col_name", "new_col_name").show(). (CAP, CAP)

Filter

Ex:- df.filter(df.columnname condition).show().

df.filter(df.address == 'india').show()

df.filter(df.name == 'manish') & (df.salary == 10000)).show().

df.filter(df.name.startswith('r')).show().

df.filter(df.name.endswith('1')).show()

df.filter(df.name.like("%al%")).show().

Distinct() & drop Duplicates()

Distinct() -> Remove the duplicates for all columns

dropDistinct() -> Remove the duplicates selected columns.

Ex:- df.distinct().show.

Sort & orderby

df.sort("column_name").show()

df.sort('column1', 'col2').show()

df.sort('col1', ascending=False).show()

df.orderBy("column").show()

df.orderBy('col1', 'col2').show()

df.orderBy("col", ascending=False).show()

GroupBy

df.groupBy("address").sum("column_name").show().

df.groupBy("address", "emp-id").sum("salary").show().

Joins

Left semi Join → only left table matching records
Left Anti Join → only left table not matching records

Ex:- emp_df.Join(dept_df, emp_df.empid == dept_df.id, 'inner').show()
Left_table.join(right_Table, condition, 'type_of_join')

Union & Union all

used to merge two or more data frames of the same schema

Ex:- df.union(df2).show
df1.unionAll(df2).show
df1.union(df2).distinct().show().

fill & fillna

used to fill NULL or 0 values.

Ex:- Sample_df.na.fill(' ').show()
sample_df.na.fill('Unknown').show()
sample_df.fillna(' ').show().
sample_df.fillna(' ', ['columns', 'col']).show()
sample_df.na.fill({"city": "Not_defined", "Population": 0}).show
For different columns with different values.

Collect

*It is useful for smaller dataset only

StructType & StructField

used to define a structure of the DataFrame

Ex:-

```
from pyspark.sql.types import StructField, StructType, IntegerType,
                              StringType.
data = [(1,'manish','usa'), (2,'mani','usa'), (1,'nish','usa')]
schema = StructType([StructField(name='id', dataType=IntegerType()),
                     StructField(name='name', dataType=StringType()),
                     StructField(name='location', dataType=StringType())])
df = spark.createDataFrame(data, schema)
df.show()
df.printSchema().
```

Pivot & unpivot

Pivot:- rows into columns

The column whose distinct values become new columns

unpivot:- column into rows

opposite to pivot

Ex:- df.groupby("product").pivot("country").sum("Amount").show()

UDF (User Defined Function)

* most useful feature of Spark SQL & Data Frame that is used to extend the pyspark build in capabilities.
* We can use UDF multiple places.
* It is most expensive operation, if no choice mean we it.

Ex:- def addone(a):
    return a+1
addone_df = udf(addone, LongType())

df.select("dr_name", "dr-id", addone_df("dr_id"). show()

Transform

It is used to perform custom transation in DataFrame.

Ex:- from pyspark.sql.functions import upper
def upper_name(df):
    return df.withColumn("name", ~~upper(df.show())~~
                upper(df.name).
df.transform(upper_name).show().

Temporary view

* Temporary views are virtual tables.
* They are not stored in catalog are only visible to the current session.
* They are automatically dropped when session ends

Ex:- df.CreateOrReplaceTempView("employee") → By using it we can perform sql queries to it also.
select * from employee

Window functions

It is used for ranking.

Ex:- from pyspark.sql.window import Window

window = Window.partitionBy ('country').orderBy ('salary').

from pyspark.sql.functions import row_number, rank, dense_rank

df.withColumn("ranking", row_number().over (window). show().

(or)

from pyspark.sql.window import Windows

from pyspark.sql.functions import rank, dense_rank, row_number

df.withColumn ('dense_rank', dense_rank().over (Window.partitionBy ('country').orderBy ('salary')).show().

for desending order -> orderBy (col('col_name').desc()).

we have to import it

date_format()

It is a powerful tool for transforming, formating date.

Ex:- from pyspark.sql.functions import *

~~df.select ('date_col').show~~

df.select ("date_col", date_format ("date_col", "yyyy/MM/dd").show

we can change this also format like dd/MM/yyyy

MM -> for Numerical month -> 1, 2, 3, 4

MMM -> for short-cut month name like -> Jan, Feb, Oct

MMMM -> for full month name -> January, February, October, December

date function

1. date_add(start_date, increase_date).

Ex:- from pyspark.sql.functions import *

df.select('date_col', date_add('date_col', 30).alias("After_add").show().

2. date_sub(start_date, no.of_date)

df.select('date_col', date_sub('date_col', 7).show().

3. datediff(end_date, start_date).

df1.select('*', datediff('new_date', 'date_col').alias('diff')).show()

4. year(col_name)

To get year of date.

df.select('*', year('date_col')).show()

5. month(col_name)

To get month. To get month name.

df.select('*', month('date_col')).show() / monthname('date_col')

6. days of year (col_name)

df.select('*', days of year('date_col')).show();

days of month

days of week

minute

hour

second

week of year

quarter.

partitionBy()

* It is useful to handle large dataset
* optimization, To increase performance.

explode

It is used to expand the data.

Ex:- from pyspark.sql.functions import *

df.select("name", explode(col("language"))).show()

for doing reverse → we use groupBy, collect_list function.

df.groupBy("name").agg(collect_list("col")).show().

Explode in SQL:-

we have to convert DataFrame to SQL so,

df.createOrReplaceTempView("sample")

Now we can use "sample" as table in SQL

Select name, explode(language) from sample.

spark.sql("query").show() by using this also we can perform sql operations in DataFrame.

dbutils

It is used for widgets

Ex:- dbutils.widgets.dropdown("time_period", 'weekly', ['weekly', 'monthly'])